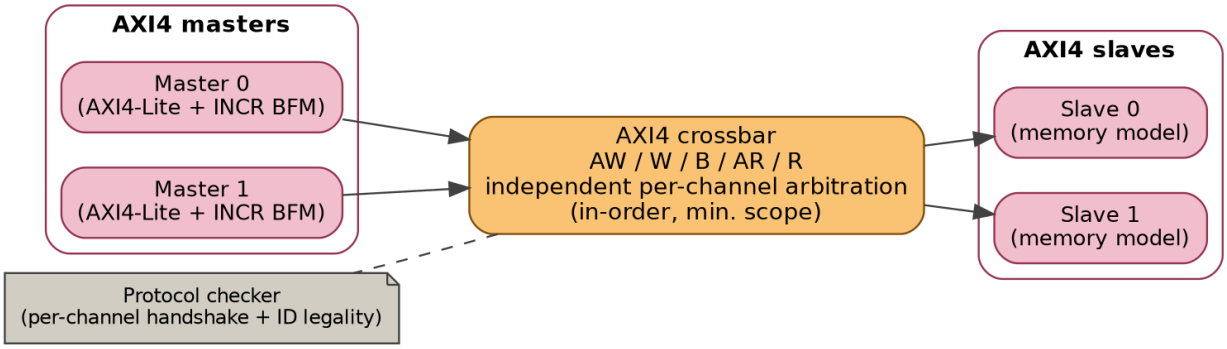
AMBA AXI4 Capstone Project

Introduction

AMBA (Advanced Microcontroller Bus Architecture) is one of the most popular interface standards for developing system-on-chip (SoC) devices. This project will involve developing semiconductor IP cores and verification IP for the AXI4 protocol. Specifications from Arm can be downloaded from here.



The above diagram shows the components of the system:

- AXI4 master bus-functional model (verification software)
- AXI4 slave bus functional model (verification software)
- Protocol checker (verification software)
- AXI4 crossbar (hardware core)

The entire design will be done in the Chisel/Scala language in accordance with the chiselWare Standard which can be downloaded from chiselWare Developers Kit on GitHub. The Developer's kit contains the complete software development in containers, including support for VS Code Dev Containers.

Requirements

Required Scope

The following are the required features of the cores.

- AXI4-Lite bundle (Chisel Bundle: independent AW, W, B, AR, R channels, each with its own valid/ready handshake; parameterized address/data width). This is the mandatory floor for the track.
- AXI4-Lite master BFM and AXI4-Lite slave BFM (parameterized memory-mapped register/memory model).
- A minimal, explicitly-scoped subset of full AXI4 beyond -Lite: burst transfers restricted to fixed-length INCR only (no WRAP/FIXED), a single ID width sufficient to tag transactions, but with only one transaction outstanding per master at a time (i.e., no reordering logic required for the required baseline).
- AXI4 (subset) master BFM and slave BFM extending the AXI4-Lite BFMs to support the INCR burst subset above.
- AXI4 protocol checker covering: per-channel handshake legality (VALID/READY rules, no combinational VALID-on-READY dependency), write-response ordering relative to W-channel completion, and basic ID consistency checks for the in-order subset.
- N x M crossbar core (minimum 2x2) with independent per-channel round-robin arbitration, address decode, and in-order response routing.
- AXI4-to-AHB-Lite bridge core, enabling interoperability
- Directed and constrained-random test suites for every block above.
- FPGA demo: a 2-master x 2-slave AXI4 (subset) crossbar instance driving representative traffic to on-chip memory-mapped slave models, with pass/fail shown on-board.

Out of Scope

The AMBA spec is quite large, so we are explicitly defining these capabilities as out of scope.

- Multiple outstanding transactions per master and ID-based out-of-order completion/reordering.
- WRAP and FIXED burst types (INCR only for the required baseline).
- Exclusive access (AXI4 exclusive monitor).
- QoS, region signaling, and user sideband signals.

Stretch goals

The following are option features that the team can add as time permits.

- Multiple outstanding transactions with proper ID-based out-of-order completion/reordering — this is the single most representative AXI4 feature and the highest-value stretch goal if time allows.
- WRAP and FIXED burst type support.
- Exclusive access support with a compliant exclusive monitor.

ChiselWare Packaging Requirements

chiselWare has very specific requirements for what a chiselWare core looks like. These are codified in the chiselWare Standard. Detailed information on designing to this specification is found in the chiselWare Developers Guide. Both can be found in the Developers Kit found on GitHub as previously mentioned.

The chiselWare standard however has not yet addressed the standardization of bus functional models (BFM). The team will therefore develop their own approach in collaboration with the sponsor with an aim that their work will one day make its way into the chiselWare Standard.

Verification Requirements

The chiselWare standard calls out standards for verification and those will apply to this project as well.

- Unit-level tests for every bundle/module using chiseltest, covering both directed and constrained-random stimulus.
- A reusable transaction-item abstraction for BFMs so the same driver supports directed tests, randomized regression
- Crossbar-specific scenarios: back-to-back transactions to the same slave, contention across multiple masters, and boundary/error cases (e.g., decode to an unmapped address).
- Per-channel handshake scenarios verifying no combinational VALID-on-READY dependency on any channel.